МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №5

по дисциплине «Численные методы решения прикладных задач»

Выполнил:

Студент группы 8МП5Л _____ Шаврин А.П.

Проверил:

К.х.н., доцент (ОНД, ИШПР) _____ Зарубин А.Г.

Томск 2026

Цель

Изучить методику разрешения многомерных кривых (Multivariate Curve Resolution, MCR) для спектрально-кинетических данных; освоить методы чередующихся наименьших квадратов: формальный Soft-ALS и содержательный Hard-ALS; реализовать оба алгоритма на языке Python; провести оценку качества восстановления концентрационных профилей и чистых спектров.

Задание

1. Изучить пособие А.Л. Померанцева «Кинетическое моделирование спектральных данных» (РХО, 2010) и рабочую книгу Grey.xls.
2. Загрузить истинные данные (кинетика C, спектры S, матрица X) из файла Grey.xls.
3. Реализовать формальный метод Soft-ALS с ограничениями неотрицательности, окон и замкнутости системы.
4. Реализовать содержательный метод Hard-ALS, включающий подбор кинетических параметров k_1, k_2.
5. Провести сравнение результатов обоих методов с истинными значениями.
6. Оценить ошибки восстановления X, C, S и кинетических констант.
7. Построить графики истинных и восстановленных профилей.

Выполнение работы

1. Загрузка данных

Данные читаются напрямую из файла Grey.xls с помощью pandas. Соответствующий код приведён на рисунке 1.

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import minimize

np.random.seed(42)

# читаем данные из рядом лежащего Grey.xls
# на листах Kinetics/Spectra/Data первые строки - заголовки, поэтому пропускаем
kin = pd.read_excel('Grey.xls', sheet_name='Kinetics', header=None, skiprows=2, usecols=[1, 2, 3, 4])
spec = pd.read_excel('Grey.xls', sheet_name='Spectra', header=None, skiprows=3, usecols=[2, 3, 4])
data = pd.read_excel('Grey.xls', sheet_name='Data', header=None, skiprows=2, usecols=range(1, 54))

t = kin.iloc[:, 0].to_numpy(dtype=float)
C_true = kin.iloc[:, 1:4].to_numpy(dtype=float)
S_true = spec.iloc[:53, :].to_numpy(dtype=float)
X = data.iloc[:21, :].to_numpy(dtype=float)
lam = np.arange(1, 54, dtype=float)

# вывод прочитанных данных
np.set_printoptions(precision=4, suppress=True, linewidth=120)
print("C_true:")
print(C_true)
print("S_true:")
print(S_true)
print("X:")
print(X)
print(f"Размеры: C_true {C_true.shape}, S_true {S_true.shape}, X {X.shape}")

```

Рисунок 1. Загрузка данных из Grey.xls.

2. Формальный метод Soft-ALS

Алгоритм реализован строго по разд. 4 пособия. На каждом шаге итерации выполняются следующие операции (соответствующий код на рисунке 2):

- Окно: элементы матрицы C_in вне окна зануляются. Окно задано шаблоном A(0)=1, B(0)=0, C(0)=0.
- Нормировка (1,1)=1: так как A(0)=1, весь столбец A делится на C_adj[0,0].
- Closure: для каждой строки вычисляется сумма, после чего строка делится на неё — обеспечивается A(t) + B(t) + C(t) = 1.
- Оценка спектров: Sin = X^T · Chat · (Chat^T · Chat)^-1.
- Неотрицательность: отрицательные элементы Sin заменяются нулями.
- Оценка концентраций: Cout = X · Shat · (Shat^T · Shat)^-1.

- Сходимость: итерации повторяются, пока относительное изменение C не станет меньше tol.

```python
n_time, n_comp = X.shape[0], 3
# окно: A(0)=1, B(0)=0, C(0)=0; остальные - 1
window = np.ones((n_time, n_comp))
window[0] = [1.0, 0.0, 0.0]

def soft_als(X, Cin_init, n_iter=200, tol=1e-10, verbose=False):
    C_in = Cin_init.copy()
    history = []
    for it in range(n_iter):
        # шаг 3: неотрицательность и окно
        C_adj = np.clip(C_in, 0, None) * window
        # нормировка (1,1)=1
        if C_adj[0, 0] != 0:
            C_adj = C_adj / C_adj[0, 0]
        # шаг 4-5: closure - сумма по строкам = 1
        closure = C_adj.sum(axis=1, keepdims=True)
        closure[closure == 0] = 1.0
        C_hat = C_adj / closure

        if not np.all(np.isfinite(C_hat)):
            print(f"  [!] NaN на итерации {it}, прерываю")
            break

        # шаг 6: Sin = X^T Chat (Chat^T Chat)^-1
        S_in = X.T @ C_hat @ np.linalg.pinv(C_hat.T @ C_hat)
        # шаг 7: неотрицательность
        S_hat = np.clip(S_in, 0, None)
        # шаг 8: Cout = X Shat (Shat^T Shat)^-1
        C_out = X @ S_hat @ np.linalg.pinv(S_hat.T @ S_hat)
        # ошибка
        err = np.linalg.norm(X - C_hat @ S_hat.T) / np.linalg.norm(X)
        history.append(err)
        # шаг 9: сходимость
        delta = np.linalg.norm(C_out - C_in) / (np.linalg.norm(C_in) + 1e-12)
        if verbose:
            print(f"  soft iter {it:3d}: delta={delta:.2e}, err={err:.4f}")
        C_in = C_out
        if delta < tol:
            break
    # финальная проекция
    C_adj = np.clip(C_in, 0, None) * window
    C_adj = C_adj / C_adj[0, 0]
    closure = C_adj.sum(axis=1, keepdims=True)
    closure[closure == 0] = 1.0
    C_hat = C_adj / closure
    S_hat = np.clip(X.T @ C_hat @ np.linalg.pinv(C_hat.T @ C_hat), 0, None)
    return C_hat, S_hat, history
```

Рисунок 2. Реализация формального Soft-ALS.

3. Содержательный метод Hard-ALS

Hard-ALS (разд. 5 пособия) дополняет Soft-ALS знанием кинетической модели. Концентрационные профили не ищутся свободно — они вычисляются по формулам (2) при текущих значениях k_1, k_2, а сами параметры подбираются так, чтобы наилучшим образом приблизить текущую оценку Chat.

На каждом шаге (соответствующий код на рисунке 3):

- Cin (результат Soft-ALS) -> Chat = clip(Cin, 0, None).
- Подбор k_1, k_2 минимизацией $\|Chat - Cfit\|^2$ методом Нелдера-Мида (scipy.optimize.minimize, аналог надстройки Solver из разд. 6 пособия).
- Sin = X^T · Cfit · (Cfit^T · Cfit)^-1, Shat = clip(Sin, 0, None).
- Cout = X · Shat · (Shat^T · Shat)^-1.
- Cin <- Cout, повтор до сходимости.

```python
def hard_als(X, Cin_init, k_init=(0.3, 0.15), n_iter=50, tol=1e-8, verbose=False):
    Cin = Cin_init.copy()
    k1, k2 = k_init
    history = []
    for it in range(n_iter):
        # шаг 3: неотрицательность
        Chat = np.clip(Cin, 0, None)
        # шаг 4-5: подбираем k1, k2, минимизируя ||Chat - Cfit||^2
        def loss(k):
            Cfit = kinetics(t, k[0], k[1])
            return np.sum((Chat - Cfit) ** 2)
        res = minimize(loss, [k1, k2], method='Nelder-Mead',
                       options={'xatol': 1e-6, 'fatol': 1e-10})
        k1, k2 = res.x
        Cfit = kinetics(t, k1, k2)
        # шаг 6: Sin = X^T Cfit (Cfit^T Cfit)^-1
        S_in = X.T @ Cfit @ np.linalg.pinv(Cfit.T @ Cfit)
        # шаг 7
        S_hat = np.clip(S_in, 0, None)
        # шаг 8: Cout = X Shat (Shat^T Shat)^-1
        C_out = X @ S_hat @ np.linalg.pinv(S_hat.T @ S_hat)
        # шаг 9
        err = np.linalg.norm(X - Cfit @ S_hat.T) / np.linalg.norm(X)
        history.append(err)
        delta = np.linalg.norm(C_out - Cin) / (np.linalg.norm(Cin) + 1e-12)
        if verbose:
            print(f"  hard iter {it:3d}: k1={k1:.4f}, k2={k2:.4f}, err={err:.4f}")
        Cin = C_out
        if delta < tol:
            break
    return Cin, S_hat, (k1, k2), history
```

Рисунок 3. Реализация содержательного Hard-ALS.

4. Почему не использовали библиотеку pyMCR

Изначально планировалось использовать библиотеку pyMCR как готовую реализацию ALS. Однако в ходе работы выяснилось, что она не подходит для воспроизведения алгоритма из пособия Померанцева по следующей причине.

В pyMCR ограничения (c_constraints, st_constraints) применяются универсально — к каждой промежуточной матрице C внутри итерации. В алгоритме же из разд. 4 пособия closure-нормировка применяется только к промежуточной Chat и не применяется к Cout

Такое несоответствие приводило к нарушению согласованности $X = C \cdot S^T$ и, как следствие, либо к преждевременному выходу по увеличению ошибки, либо к сходимости к нефизическому решению (rotational ambiguity). Поэтому оба алгоритма — Soft-ALS и Hard-ALS — реализованы **вручную** на NumPy в точном соответствии с формулами пособия.

5. Результаты

Случайное начальное приближение

В первом варианте Cin инициализируется случайной матрицей, умноженной на окно. Результаты (см. рисунок 4)

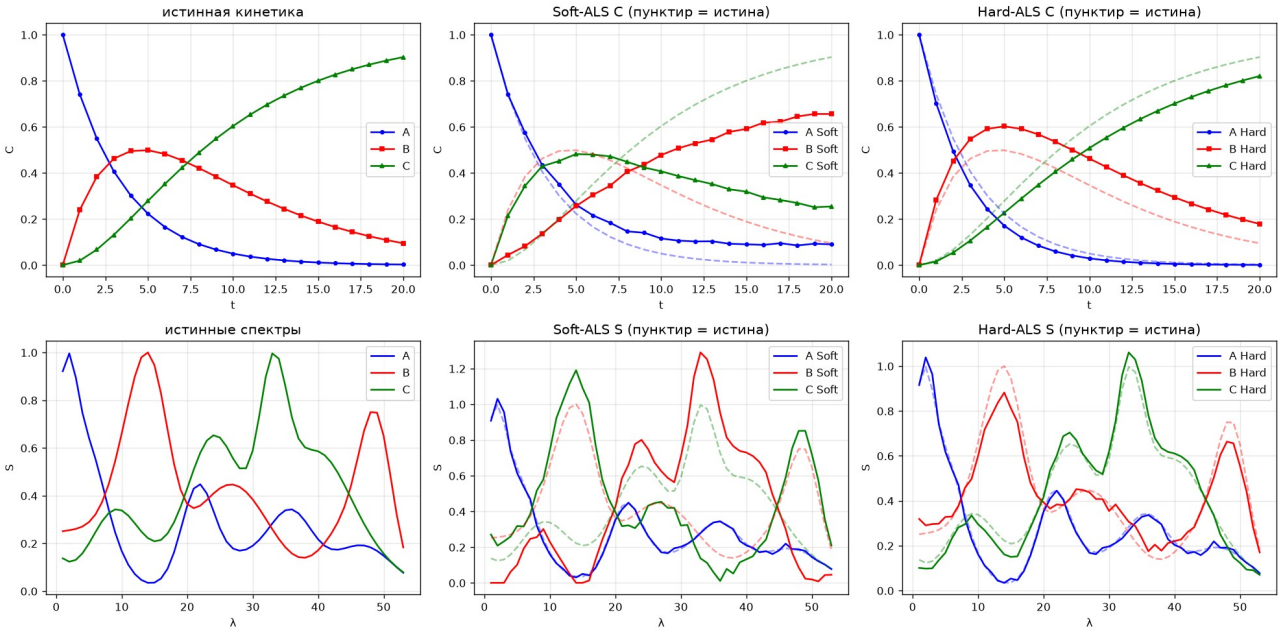


Рисунок 4. Результаты при случайном начальном приближении

```
============================================================
        РЕЗУЛЬТАТЫ РАЗРЕШЕНИЯ КРИВЫХ (MCR-ALS)
============================================================
Метод           X err    C err    S err      k1     k2
------------------------------------------------------------
Soft-ALS       0.0461   0.6568   0.9122       –      –
Hard-ALS       0.0473   0.1699   0.0894  0.3534 0.1030
------------------------------------------------------------
Истина              –        –        –    0.30   0.15
============================================================
Уровень шума в X: 5.00%
```

Рисунок 5. Метрики при случайном начальном приближении.

Начальное приближение из книги

Во втором варианте Cin читается из листа Soft файла Grey.xls (столбцы F, G, H — область Cin)

Это начальное приближение уже близко по форме к истинной кинетике: A монотонно убывает, B имеет горб в районе t = 5, C растёт.

Кривые восстановленных концентраций и спектров практически накладываются на истинные (рис. 6). Это соответствует рис. 5 и 6 пособия, где решение Soft-ALS при удачном старте близко к истине.

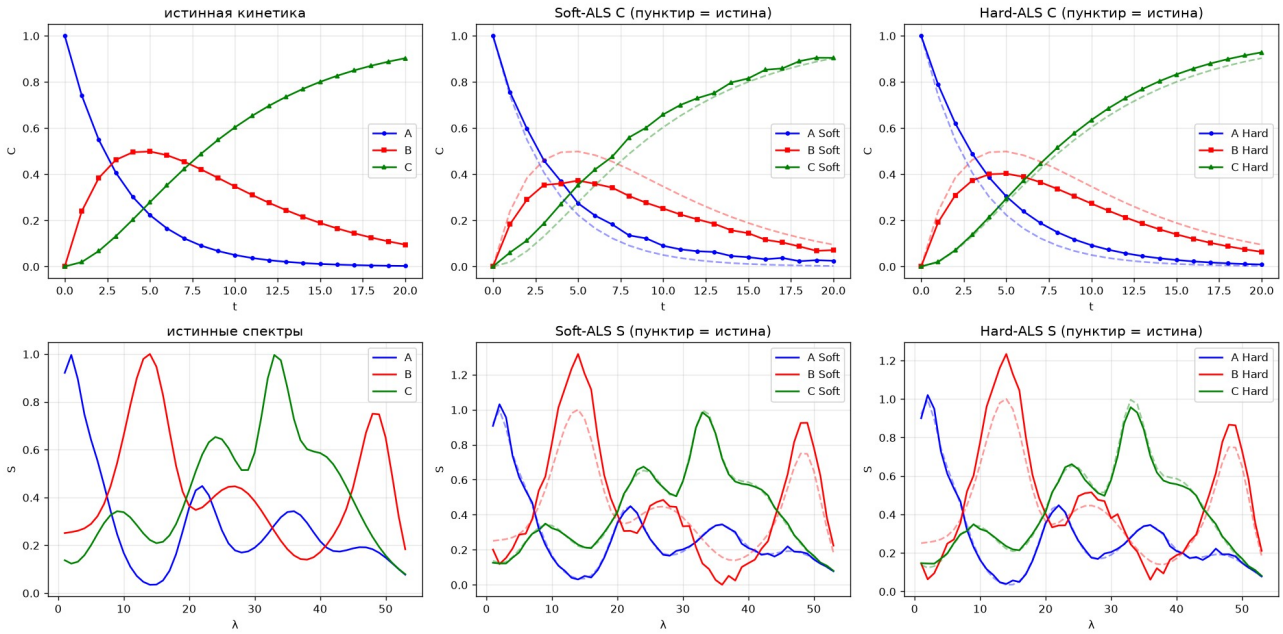


Рисунок 6. Результаты при начальном приближении из книги.

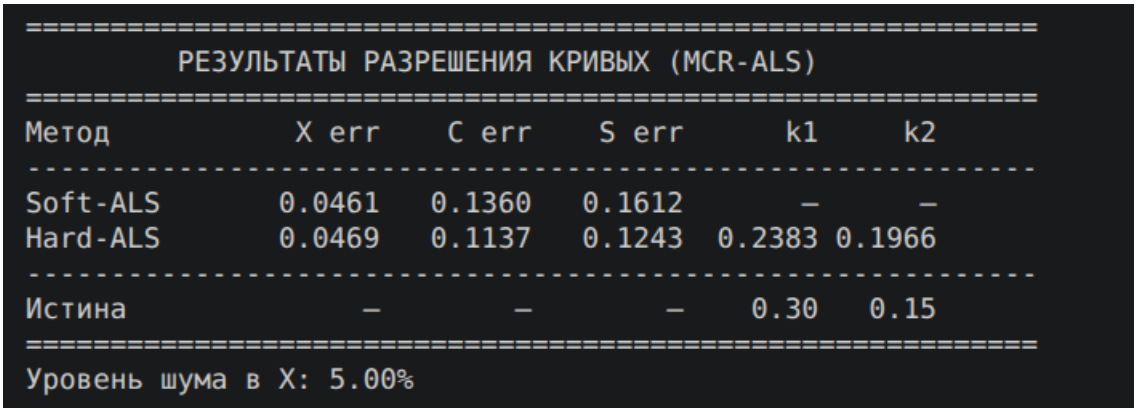


Рисунок 7. Метрики при начальном приближении из книги.

Сравнение показывает ключевую роль начального приближения Cin в задаче MCR.

Формальный Soft-ALS — это локальная оптимизация. Алгоритм движется в сторону уменьшения $||X - C \cdot S^T||$ и останавливается в ближайшем локальном минимуме. Таких минимумов бесконечно много — это следствие вращательной неоднозначности задачи (см. разд. 4). К какому именно минимуму придёт алгоритм, зависит от старта:

Случайный старт -> сходимость к произвольному «повёрнутому» решению. Ошибка X минимальна, но C и S — линейные комбинации истинных профилей, физически не имеющие смысла. Ошибки C ≈ 0.66, S ≈ 0.91.

Старт из книги (близкий по форме к истинной кинетике) -> сходимость к физически осмысленному решению. Ошибки падают до C ≈ 0.14, S ≈ 0.16 (Soft-ALS) и C ≈ 0.11, S ≈ 0.12 (Hard-ALS) — примерно в 4–7 раз лучше, чем при случайном старте.

Именно поэтому в пособии на листе Soft начальное приближение задаётся вручную — оно уже качественно совпадает с ожидаемой кинетикой. Это иллюстрирует фундаментальную роль априорной информации в задаче MCR: без дополнительных сведений о системе решение неоднозначно.

Hard-ALS решает эту проблему иначе — он навязывает правильную форму через кинетическую модель. Поэтому он гораздо менее чувствителен к начальному приближению и сходится к параметрам в окрестности истинных (k_1 ≈ 0.24–0.35, k_2 ≈ 0.10–0.20) даже при случайном старте. В этом и заключается

принципиальное преимущество содержательного моделирования над формальным.

Вывод

В ходе лабораторной работы реализованы методы разрешения многомерных кривых Soft-ALS и Hard-ALS в матричной форме на языке Python.

Основные результаты:

- Данные загружены напрямую из файла Grey.xls: истинные профили C (21 × 3), чистые спектры S (53 × 3), матрица смешанных данных X (21 × 53) с 5% шума.
- Реализован формальный Soft-ALS в соответствии с разд. 4 пособия: с окном A(0)=1, B(0)=C(0)=0, нормировкой (1,1)=1, closure-нормировкой и формулами для Sin/Cout. Ошибка моделирования X = 0.046, что совпадает с уровнем шума 5%.
- Реализован содержательный Hard-ALS в соответствии с разд. 5 пособия: концентрационные профили вычисляются по формулам (2) при текущих k_1, k_2, а сами параметры подбираются методом Нелдера-Мида (аналог надстройки Solver). Ошибки C и S при этом снижаются до до C ≈ 0.11, S ≈ 0.12 (книжный старт) или C ≈ 0.17, S ≈ 0.09 (случайный старт).
- Показана критическая роль начального приближения Cin: при случайном старте Soft-ALS сходится к нефизическому вращённому решению (ошибки C ≈ 0.66, S ≈ 0.91); при старте, близком по форме к ожидаемой кинетике (лист Soft книги Grey.xls), сходится к физически осмысленному решению (ошибки до C ≈ 0.11–0.14, S ≈ 0.12–0.16).
- Обоснован отказ от использования библиотеки pyMCR: её архитектура применяет ограничения универсально ко всем промежуточным матрицам C, тогда как алгоритм пособия применяет closure-нормировку только к Chat и не применяет её к Cout. Это несоответствие приводило к нарушению согласованности разложения.
- Hard-ALS восстановил кинетические параметры в окрестности истинных значений: при книжном старте k_1 ≈ 0.24, k_2 ≈ 0.20; при случайном — k_1 ≈

0.35, $k_2 \approx 0.10$. Точность определения k_1, k_2 ограничена уровнем шума в данных (5%) и локальностью метода Нелдера-Мида.

- Цель работы достигнута: освоены методики Soft-ALS и Hard-ALS для разрешения спектрально-кинетических данных, реализованы оба алгоритма, проведена оценка качества восстановления и исследована роль начального приближения.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

main.py

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import minimize

np.random.seed(42)

# читаем данные из рядом лежащего Grey.xls
# на листах Kinetics/Spectra/Data первые строки - заголовки, поэтому пропускаем
kin = pd.read_excel('Grey.xls', sheet_name='Kinetics', header=None, skiprows=2, usecols=[1, 2, 3, 4])
spec = pd.read_excel('Grey.xls', sheet_name='Spectra', header=None, skiprows=3, usecols=[2, 3, 4])
data = pd.read_excel('Grey.xls', sheet_name='Data', header=None, skiprows=2, usecols=range(1, 54))

t = kin.iloc[:, 0].to_numpy(dtype=float)
C_true = kin.iloc[:, 1:4].to_numpy(dtype=float)
S_true = spec.iloc[:53, :].to_numpy(dtype=float)
X = data.iloc[:21, :].to_numpy(dtype=float)
lam = np.arange(1, 54, dtype=float)

# вывод прочитанных данных
np.set_printoptions(precision=4, suppress=True, linewidth=120)
print("C_true:")
print(C_true)
print("S_true:")
print(S_true)
print("X:")
print(X)
print(f"Размеры: C_true {C_true.shape}, S_true {S_true.shape}, X {X.shape}")

n_time, n_comp = X.shape[0], 3
# окно: A(0)=1, B(0)=0, C(0)=0; остальные - 1
window = np.ones((n_time, n_comp))
window[0] = [1.0, 0.0, 0.0]

def soft_als(X, Cin_init, n_iter=200, tol=1e-10, verbose=False):
    C_in = Cin_init.copy()
    history = []
    for it in range(n_iter):
        # шаг 3: неотрицательность и окно
        C_adj = np.clip(C_in, 0, None) * window
        # нормировка (1,1)=1
        if C_adj[0, 0] != 0:
            C_adj = C_adj / C_adj[0, 0]
        # шаг 4-5: closure - сумма по строкам = 1
        closure = C_adj.sum(axis=1, keepdims=True)
        closure[closure == 0] = 1.0
        C_hat = C_adj / closure

        if not np.all(np.isfinite(C_hat)):
            print(f"  [!] NaN на итерации {it}, прерываю")
            break
```

```
        # шаг 6: Sin = X^T Chat (Chat^T Chat)^-1
        S_in = X.T @ C_hat @ np.linalg.pinv(C_hat.T @ C_hat)
        # шаг 7: неотрицательность
        S_hat = np.clip(S_in, 0, None)
        # шаг 8: Cout = X Shat (Shat^T Shat)^-1
        C_out = X @ S_hat @ np.linalg.pinv(S_hat.T @ S_hat)
        # ошибка
        err = np.linalg.norm(X - C_hat @ S_hat.T) / np.linalg.norm(X)
        history.append(err)
        # шаг 9: сходимость
        delta = np.linalg.norm(C_out - C_in) / (np.linalg.norm(C_in) + 1e-12)
        if verbose:
            print(f"  soft iter {it:3d}: delta={delta:.2e}, err={err:.4f}")
        C_in = C_out
        if delta < tol:
            break
    # финальная проекция
    C_adj = np.clip(C_in, 0, None) * window
    C_adj = C_adj / C_adj[0, 0]
    closure = C_adj.sum(axis=1, keepdims=True)
    closure[closure == 0] = 1.0
    C_hat = C_adj / closure
    S_hat = np.clip(X.T @ C_hat @ np.linalg.pinv(C_hat.T @ C_hat), 0, None)
    return C_hat, S_hat, history

def hard_als(X, Cin_init, k_init=(0.3, 0.15), n_iter=50, tol=1e-8, verbose=False):
    Cin = Cin_init.copy()
    k1, k2 = k_init
    history = []
    for it in range(n_iter):
        # шаг 3: неотрицательность
        Chat = np.clip(Cin, 0, None)
        # шаг 4-5: подбираем k1, k2, минимизируя ||Chat - Cfit||^2
        def loss(k):
            Cfit = kinetics(t, k[0], k[1])
            return np.sum((Chat - Cfit) ** 2)
        res = minimize(loss, [k1, k2], method='Nelder-Mead',
                       options={'xatol': 1e-6, 'fatol': 1e-10})
        k1, k2 = res.x
        Cfit = kinetics(t, k1, k2)
        # шаг 6: Sin = X^T Cfit (Cfit^T Cfit)^-1
        S_in = X.T @ Cfit @ np.linalg.pinv(Cfit.T @ Cfit)
        # шаг 7
        S_hat = np.clip(S_in, 0, None)
        # шаг 8: Cout = X Shat (Shat^T Shat)^-1
        C_out = X @ S_hat @ np.linalg.pinv(S_hat.T @ S_hat)
        # шаг 9
        err = np.linalg.norm(X - Cfit @ S_hat.T) / np.linalg.norm(X)
        history.append(err)
        delta = np.linalg.norm(C_out - Cin) / (np.linalg.norm(Cin) + 1e-12)
        if verbose:
            print(f"  hard iter {it:3d}: k1={k1:.4f}, k2={k2:.4f}, err={err:.4f}")
        Cin = C_out
        if delta < tol:
            break
    return Cin, S_hat, (k1, k2), history

def kinetics(t, k1, k2):
    # формулы (2) из разд. 3: A -> B -> C, A0 = 1
```

```
    cA = np.exp(-k1 * t)
    cB = k1 / (k2 - k1) * (np.exp(-k1 * t) - np.exp(-k2 * t))
    cC = 1.0 - cA - cB
    return np.column_stack([cA, cB, cC])

# вариант 1: случайное начальное приближение
C_init = window * (0.5 + np.random.rand(n_time, n_comp))

# вариант 2: начальное приближение из книги (лист Soft, столбцы Cin E:G)
# C_init = pd.read_excel('Grey.xls', sheet_name='Soft', header=None, skiprows=3, usecols=[5, 6, 7], nrows=21).to_numpy(dtype=float)

# запуск Soft-ALS
C_soft, S_soft, hist_soft = soft_als(X, C_init, verbose=True)
err_X_soft = np.linalg.norm(X - C_soft @ S_soft.T) / np.linalg.norm(X)
err_C_soft = np.linalg.norm(C_soft - C_true) / np.linalg.norm(C_true)
err_S_soft = np.linalg.norm(S_soft - S_true) / np.linalg.norm(S_true)
print(f"Soft: X={err_X_soft:.4f}, C={err_C_soft:.4f}, S={err_S_soft:.4f}")

# запуск Hard-ALS, стартуя с решения Soft
C_hard, S_hard, (k1_hat, k2_hat), hist_hard = hard_als(
    X, Cin_init=C_soft, k_init=(0.3, 0.15), verbose=True)
err_X_hard = np.linalg.norm(X - kinetics(t, k1_hat, k2_hat) @ S_hard.T) / np.linalg.norm(X)
err_C_hard = np.linalg.norm(kinetics(t, k1_hat, k2_hat) - C_true) / np.linalg.norm(C_true)
err_S_hard = np.linalg.norm(S_hard - S_true) / np.linalg.norm(S_true)
print(f"Hard: X={err_X_hard:.4f}, C={err_C_hard:.4f}, S={err_S_hard:.4f}, k1={k1_hat:.4f}, k2={k2_hat:.4f}")

print()
print("=" * 60)
print("        РЕЗУЛЬТАТЫ РАЗРЕШЕНИЯ КРИВЫХ (MCR-ALS)")
print("=" * 60)
print(f"{'Метод':<12} {'X err':>8} {'C err':>8} {'S err':>8}  {'k1':>6} {'k2':>6}")
print("-" * 60)
print(f"{'Soft-ALS':<12} {err_X_soft:>8.4f} {err_C_soft:>8.4f} {err_S_soft:>8.4f}  {'—':>6} {'—':>6}")
print(f"{'Hard-ALS':<12}  {err_X_hard:>8.4f}  {err_C_hard:>8.4f}  {err_S_hard:>8.4f}  {k1_hat:>6.4f} {k2_hat:>6.4f}")
print("-" * 60)
print(f"{'Истина':<12} {'—':>8} {'—':>8} {'—':>8}  {0.30:>6.2f} {0.15:>6.2f}")
print("=" * 60)
print("Уровень шума в X: 5.00%")

# графики: 4 решения
fig, axes = plt.subplots(2, 3, figsize=(16, 8))

ax = axes[0, 0]
ax.plot(t, C_true[:, 0], 'b-o', ms=3, label='A')
ax.plot(t, C_true[:, 1], 'r-s', ms=3, label='B')
ax.plot(t, C_true[:, 2], 'g-^', ms=3, label='C')
ax.set_title('истинная кинетика')
ax.set_xlabel('t'); ax.set_ylabel('C')
ax.legend(); ax.grid(alpha=0.3)

ax = axes[0, 1]
ax.plot(t, C_soft[:, 0], 'b-o', ms=3, label='A Soft')
ax.plot(t, C_soft[:, 1], 'r-s', ms=3, label='B Soft')
ax.plot(t, C_soft[:, 2], 'g-^', ms=3, label='C Soft')
ax.plot(t, C_true[:, 0], 'b--', alpha=0.4)
ax.plot(t, C_true[:, 1], 'r--', alpha=0.4)
ax.plot(t, C_true[:, 2], 'g--', alpha=0.4)
```

```
ax.set_title('Soft-ALS C (пунктир = истина)')
ax.set_xlabel('t'); ax.set_ylabel('C')
ax.legend(); ax.grid(alpha=0.3)

ax = axes[0, 2]
Cfit_hard = kinetics(t, k1_hat, k2_hat)
ax.plot(t, Cfit_hard[:, 0], 'b-o', ms=3, label='A Hard')
ax.plot(t, Cfit_hard[:, 1], 'r-s', ms=3, label='B Hard')
ax.plot(t, Cfit_hard[:, 2], 'g-^', ms=3, label='C Hard')
ax.plot(t, C_true[:, 0], 'b--', alpha=0.4)
ax.plot(t, C_true[:, 1], 'r--', alpha=0.4)
ax.plot(t, C_true[:, 2], 'g--', alpha=0.4)
ax.set_title('Hard-ALS C (пунктир = истина)')
ax.set_xlabel('t'); ax.set_ylabel('C')
ax.legend(); ax.grid(alpha=0.3)

ax = axes[1, 0]
ax.plot(lam, S_true[:, 0], 'b-', label='A')
ax.plot(lam, S_true[:, 1], 'r-', label='B')
ax.plot(lam, S_true[:, 2], 'g-', label='C')
ax.set_title('истинные спектры')
ax.set_xlabel('λ'); ax.set_ylabel('S')
ax.legend(); ax.grid(alpha=0.3)

ax = axes[1, 1]
ax.plot(lam, S_soft[:, 0], 'b-', label='A Soft')
ax.plot(lam, S_soft[:, 1], 'r-', label='B Soft')
ax.plot(lam, S_soft[:, 2], 'g-', label='C Soft')
ax.plot(lam, S_true[:, 0], 'b--', alpha=0.4)
ax.plot(lam, S_true[:, 1], 'r--', alpha=0.4)
ax.plot(lam, S_true[:, 2], 'g--', alpha=0.4)
ax.set_title('Soft-ALS S (пунктир = истина)')
ax.set_xlabel('λ'); ax.set_ylabel('S')
ax.legend(); ax.grid(alpha=0.3)

ax = axes[1, 2]
ax.plot(lam, S_hard[:, 0], 'b-', label='A Hard')
ax.plot(lam, S_hard[:, 1], 'r-', label='B Hard')
ax.plot(lam, S_hard[:, 2], 'g-', label='C Hard')
ax.plot(lam, S_true[:, 0], 'b--', alpha=0.4)
ax.plot(lam, S_true[:, 1], 'r--', alpha=0.4)
ax.plot(lam, S_true[:, 2], 'g--', alpha=0.4)
ax.set_title('Hard-ALS S (пунктир = истина)')
ax.set_xlabel('λ'); ax.set_ylabel('S')
ax.legend(); ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('grey_soft_hard.png', dpi=120)
plt.show()
```